
Summary: Tile Engine → Dispatcher Migration — Corrected Direction

Author: Muhammed Öztürk

Date: 2026-06-03

For review by: Vidyasagar, Thrupti, Yaswanth

One-Sentence Goal

Replace Tile Engine's CMake-based per-config code generation with the Dispatcher's Python-driven codegen, prove feature coverage is equivalent, then delete the Tile Engine machinery.

What the Project Is NOT

The project is **not** building a translator layer that sits between TE and the Dispatcher. It is **not** proving that a custom C++ harness agrees with a TE binary. Those are useful intermediate verification tools, but they are not the deliverable.

What the Project IS

Step 1 — Replace the codegen mechanism

Tile Engine today generates kernels like this:

```
CMakeLists.txt
→ gemm_universal_instance_builder.py
→ hundreds of generated C++ files
→ one compiled binary per config
```

The Dispatcher generates kernels like this:

```
Python script
→ unified_gemm_codegen.py
→ handful of generated C++ files
→ one shared library (.so) with all kernels
```

The job is to drive `unified_gemm_codegen.py` (which already exists in the Dispatcher) to cover every dtype / layout / pipeline / architecture combination that TE's CMake build covered today. No new codegen tool is needed — only a Python driver that feeds the right configs to the one that already exists.

Step 2 — Prove coverage (parity)

Run the same GEMM problem sizes through:

- TE binary (old path)
- `DispatcherLib.run_gemm()` (new path)

Assert: outputs match (numerical parity) and performance is within $\pm 2\%$ (TFLOPS parity). This is the proof that the new path is a safe replacement.

Step 3 — Delete TE

Once parity is confirmed across all dtypes and layouts, remove:

- TE's `CMakeLists.txt` codegen rules
- TE's per-config generated C++ files
- TE's individual benchmark binaries

What remains: Dispatcher's `unified_gemm_codegen.py`, the `.so`, and a thin Python interface (`DispatcherLib`) that callers use instead of TE binaries.

Architectural Direction (from Vidyasagar)

Do NOT make the Dispatcher look like Tile Engine.

Make Tile Engine conform to the Dispatcher model.

This means: the Dispatcher's vocabulary, config format, and Python interface are the canonical ones. Tile Engine adapts to them — not the other way around.

What the Current PR Built (and What to Keep)

Current PR artifact	Status
<code>te_to_dispatcher.py</code> — field mapping between TE and Dispatcher	Keep — useful for migration verification
<code>check_identifier_parity.py</code> — 220 tests proving kernel names match	Keep — critical regression guard
<code>drive_codegen.py</code> — drives <code>unified_gemm_codegen.py</code>	Keep with refactor — this is the core mechanism
<code>harness.cpp</code> — custom single-kernel C++ runner	Replace with <code>DispatcherLib.run_gemm()</code>
<code>check_parity.py</code> — 3-stage parity orchestrator	Keep — extend to use <code>DispatcherLib</code>

The core issue with the current PR: parity was proven via a custom `harness.cpp` instead of `DispatcherLib`. Callers in production will use `DispatcherLib`, not the harness. Parity must be proven through the actual production interface.

Phase 1 Plan (one config, end-to-end)

- Choose one representative config: `fp16 / rcr / compv3 / intrawave`
- Drive `unified_gemm_codegen.py` to generate that kernel
- Load it via `DispatcherLib.load()` — no custom harness
- Run the same problem through TE binary and `DispatcherLib.run_gemm()`
- Assert numerical parity ($\text{rtol}=1\text{e-}2$) and TFLOPS parity ($\leq 2\%$)

Done when: one config passes both checks via `DispatcherLib`.

Phase 2 Plan (full sweep, delete TE)

- Extend to full `fp16/rcr` matrix (all tile/warp/pipeline combos)
- Expand to `bf16` and `fp8`
- Build comparison report (HTML/Markdown, one row per config)
- Confirm $\geq 99\%$ pass rate and $\leq 2\%$ TFLOPS delta across all configs
- Delete TE codegen machinery

Done when: TE binaries are deleted and `DispatcherLib` is the only path.

Questions Requiring Team Input Before Proceeding

- Which single config should be the Phase 1 target?
- Is `DispatcherLib` the intended production interface, or is there a different

Python entry point planned?

- Which TE config files (JSON) define the full set to be covered in Phase 2?
- Is the `Dispatcher` .so already built on a CI machine, or does Phase 1 require

a fresh CMake build first?